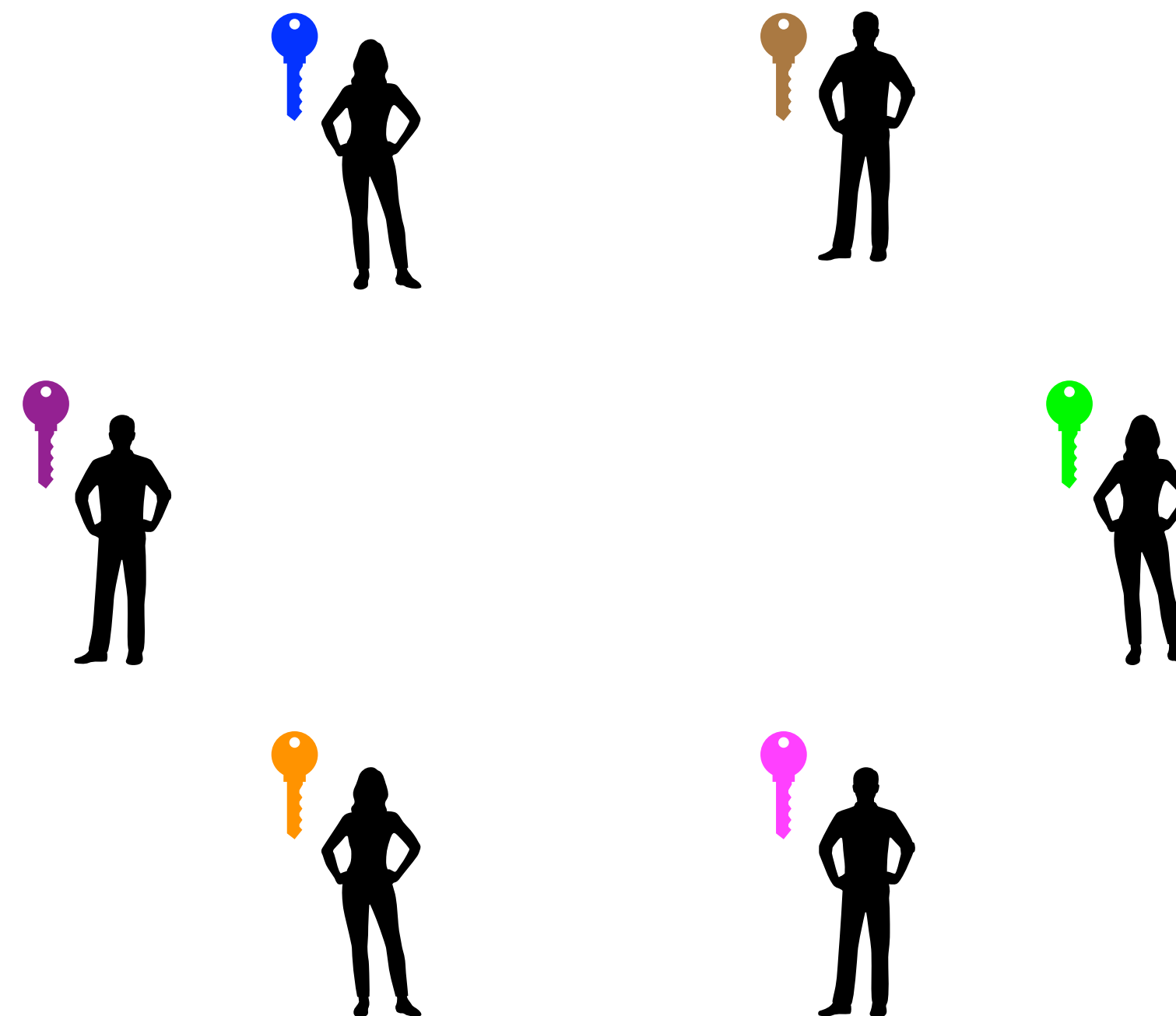


A Lattice-Based IND-CCA Threshold KEM from the BCHK+ Transform

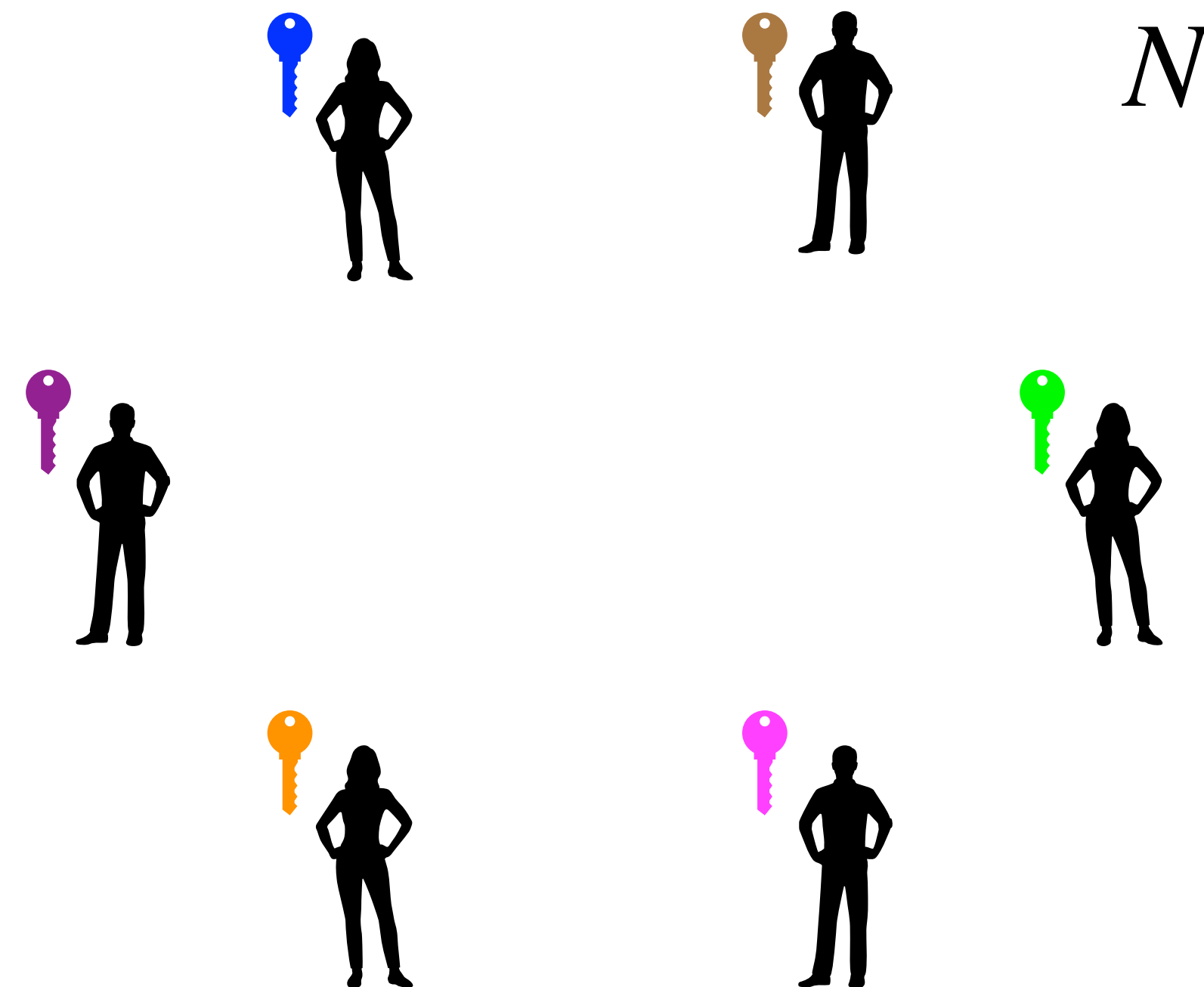
By Sasha Lapiha and Thomas Prest



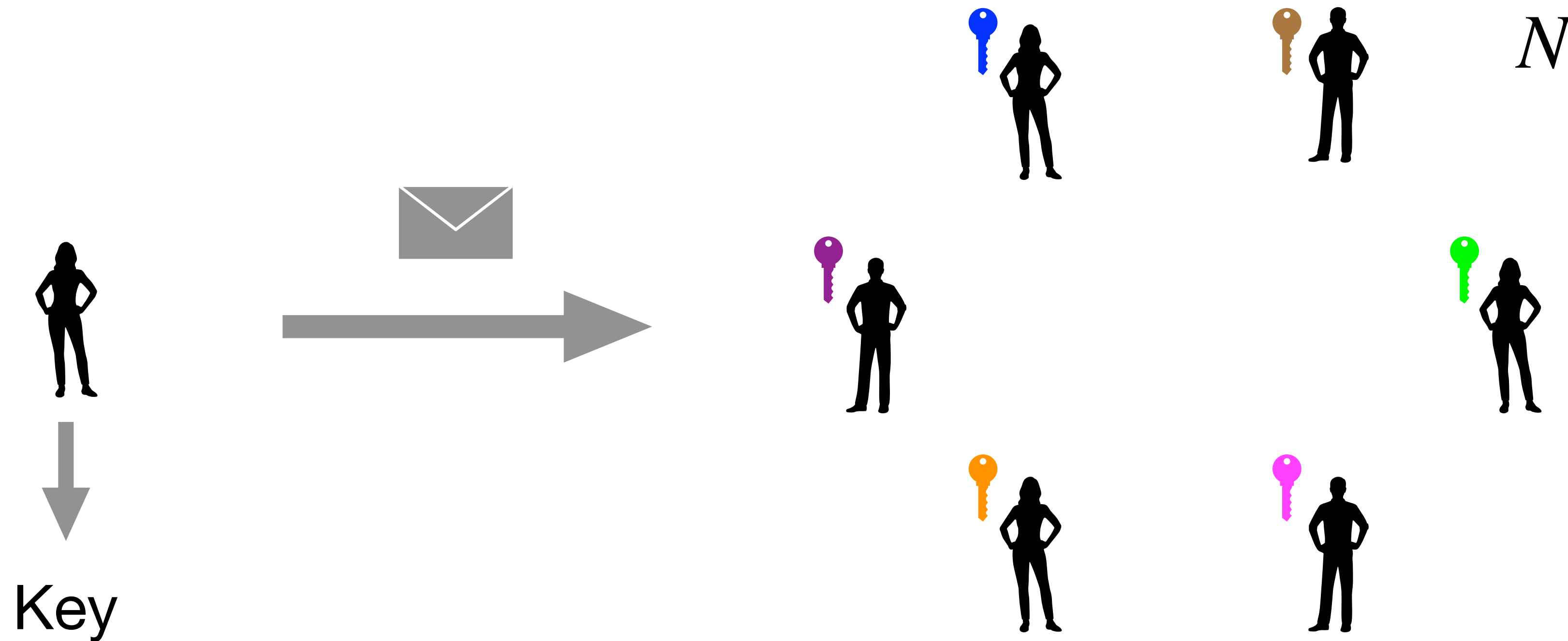
Threshold Cryptography



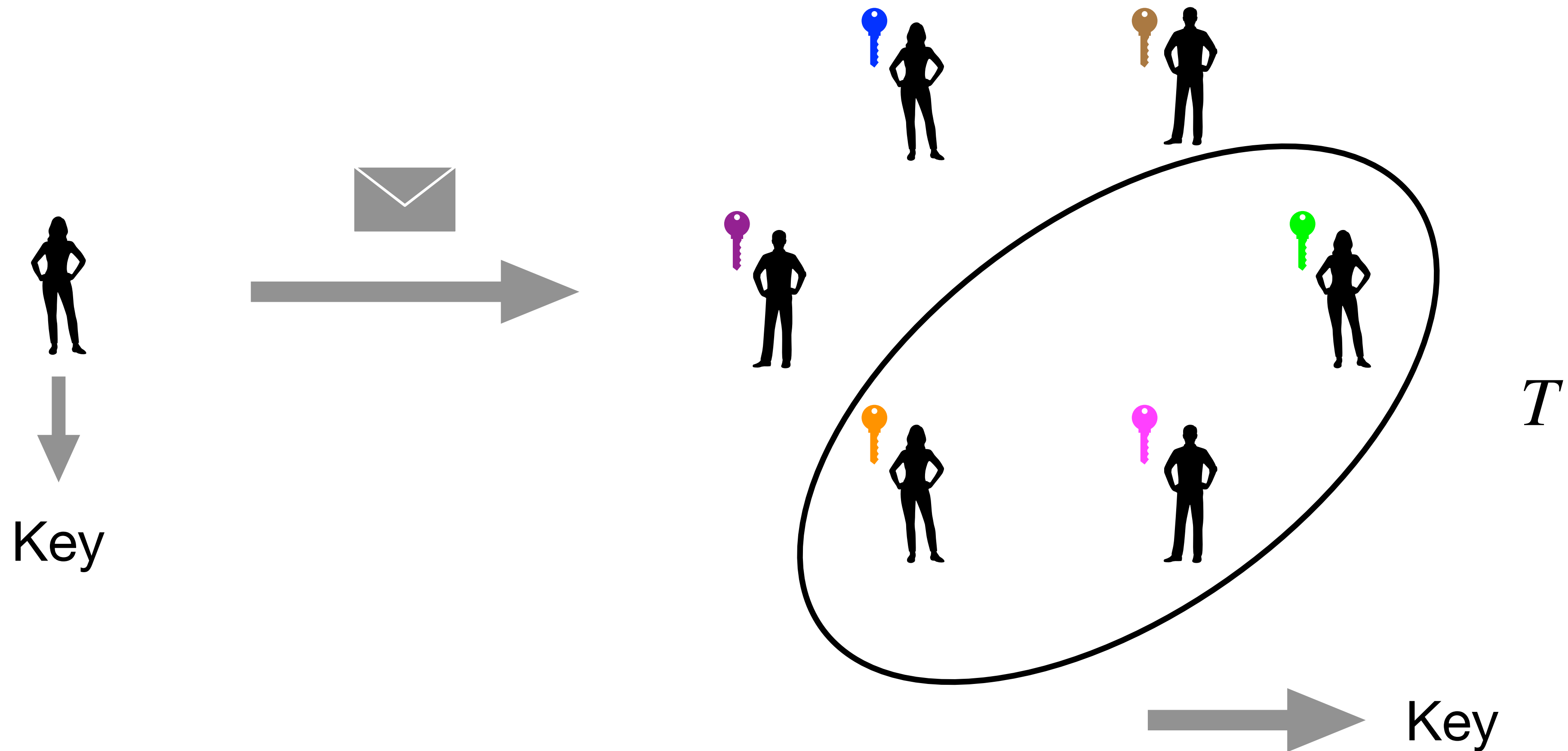
Threshold Decapsulation (TKEM)



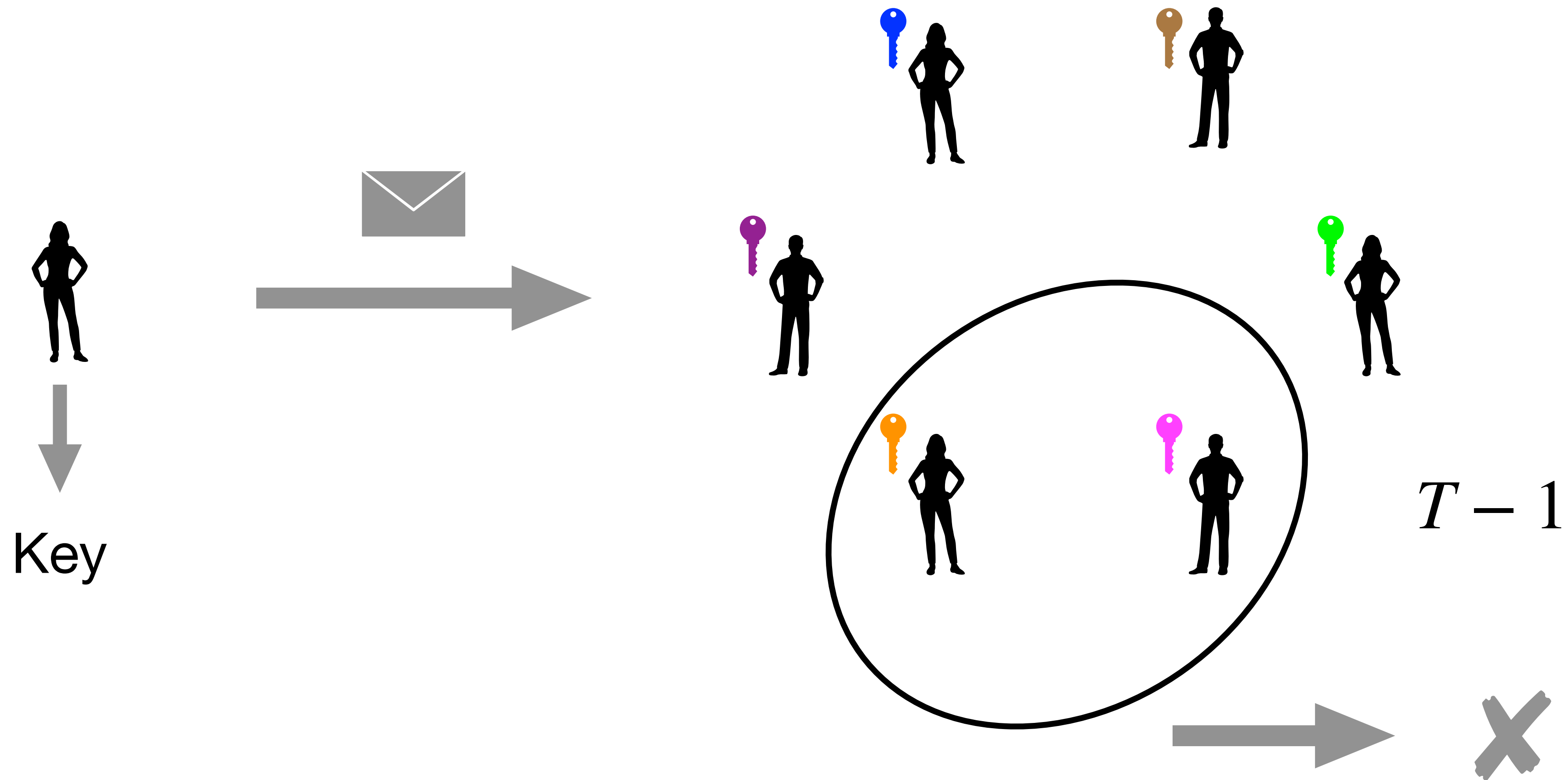
Threshold Decapsulation



Threshold Decapsulation



Threshold Decapsulation



Obtaining CCA security

Previous work

Reference	Security	Decrypt	ek	ct	Rounds	Robust	Tools
IND-CCA TKEM							
Boneh et al. [12]	Selective, UC	N/A	N/A	N/A	1	Yes	FHE
Kraitsberg et al. [41]	Heuristic	4.342 sec.	4	4	≥ 114	No	MPC
Devevey et al. [24]	Adaptive	N/A	N/A	N/A	1	IA	NIZK
Cong et al. [20]	Selective, UC	4.99 sec.	N/A	N/A	136498	IA	MPC
This work	Selective	N/A	50	450	3	No	Generic

Related work comparison for TKEM. Sizes of |ek| and |ct| are given in KiB. N/A = Not Available, UC = Universal Composability, IA = Identifiable Aborts

Plan

1. The Transform: BCHK+
2. Components: Lattice-Based Threshold IBE
3. Security: Coset-Hint-RLWE assumption (standard)
4. Parameters and Future work

BCHK+ Transform

BCHK Transform [CHK04,BBH06,BBH07]

$\text{TPKE.Keygen}(1^\kappa)$

$(\{\text{dk}_i\}_{i \in [N]}, \text{pk}) \leftarrow \text{TIBE.Setup}(1^\kappa)$

return $(\{\text{dk}_i\}_{i \in [N]}, \text{pk})$

BCHK Transform [CHK04,BBH06,BBH07]

TPKE.Keygen(1^κ)

$(\{\text{dk}_i\}_{i \in [N]}, \text{pk}) \leftarrow \text{TIBE.Setup}(1^\kappa)$
return $(\{\text{dk}_i\}_{i \in [N]}, \text{pk})$

TPKE.Encrypt(pk)

$(\text{sk}, \text{vk}) \leftarrow \text{SIG.Keygen}(1^\kappa)$
 $\text{ct} = \text{TIBE.Encrypt}(\text{pk}, \text{id} = \text{vk}, \text{msg})$
 $\text{sig} \leftarrow \text{SIG.Sign}(\text{sk}, \text{ct})$
return $(\text{ct}, \text{vk}, \text{sig})$

BCHK Transform [CHK04,BBH06,BBH07]

TPKE.Keygen(1^κ)

$(\{dk_i\}_{i \in [N]}, pk) \leftarrow \text{TIBE.Setup}(1^\kappa)$
return $(\{dk_i\}_{i \in [N]}, pk)$

TPKE.Decrypt($dk_i, (ct, vk, sig)$)

assert $\text{SIG.Verify}((vk, ct, sig)) = 1$
 $contrib_i = \text{TIBE.Extract}(dk_i, id = vk)$
return $contrib_i$

TPKE.Encrypt(pk)

$(sk, vk) \leftarrow \text{SIG.Keygen}(1^\kappa)$
 $ct = \text{TIBE.Encrypt}(pk, id = vk, msg)$
 $sig \leftarrow \text{SIG.Sign}(sk, ct)$
return (ct, vk, sig)

TPKE.Combine($\{contrib_i\}_{i \in \text{act}}, (ct, vk, sig)$)

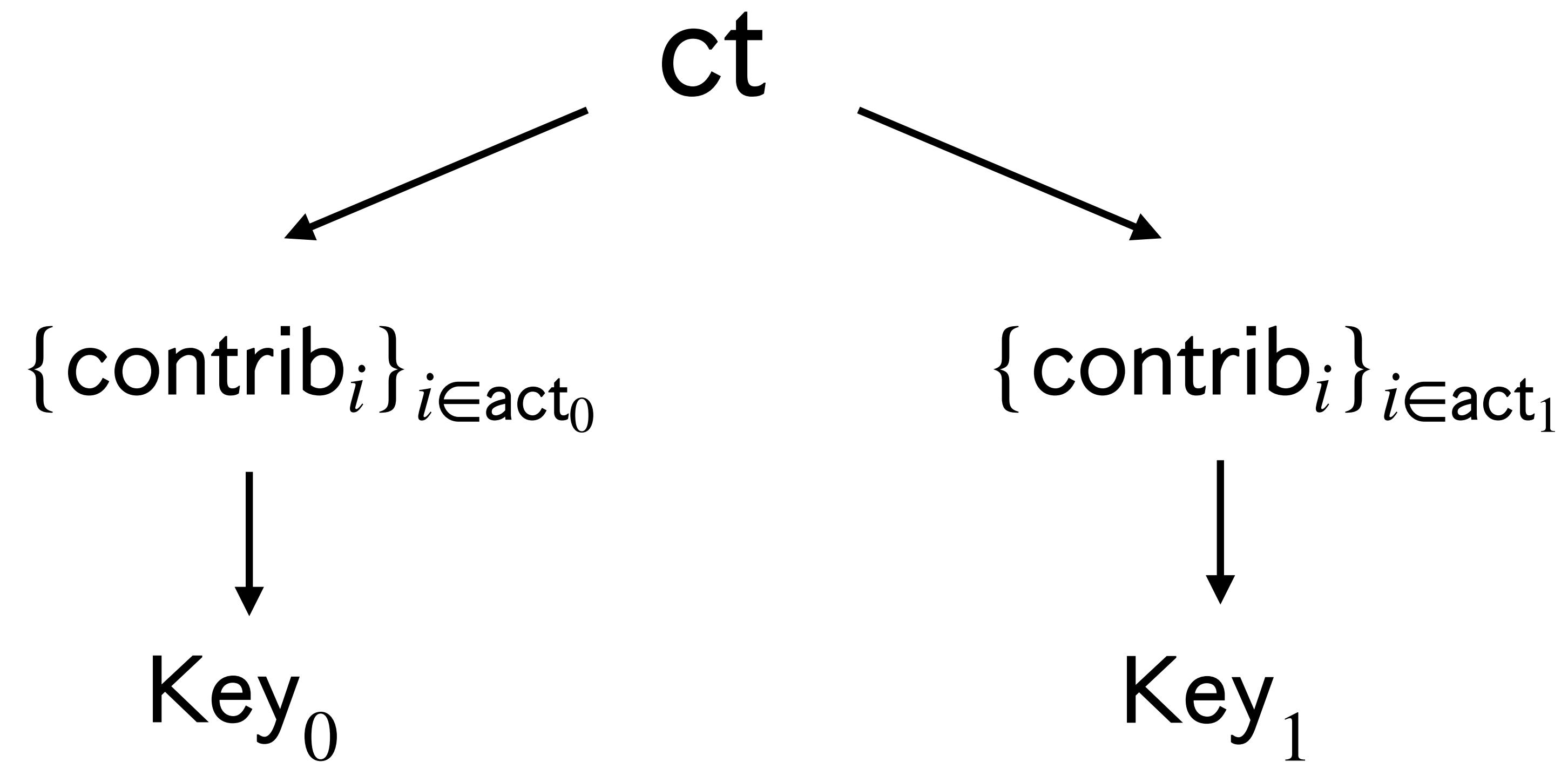
assert $\text{SIG.Verify}((vk, ct, sig)) = 1$
 $msg = \text{TIBE.Combine}(ct, vk, \text{act}, \{contrib_i\}_i)$
return msg

Decapsulation Consistency

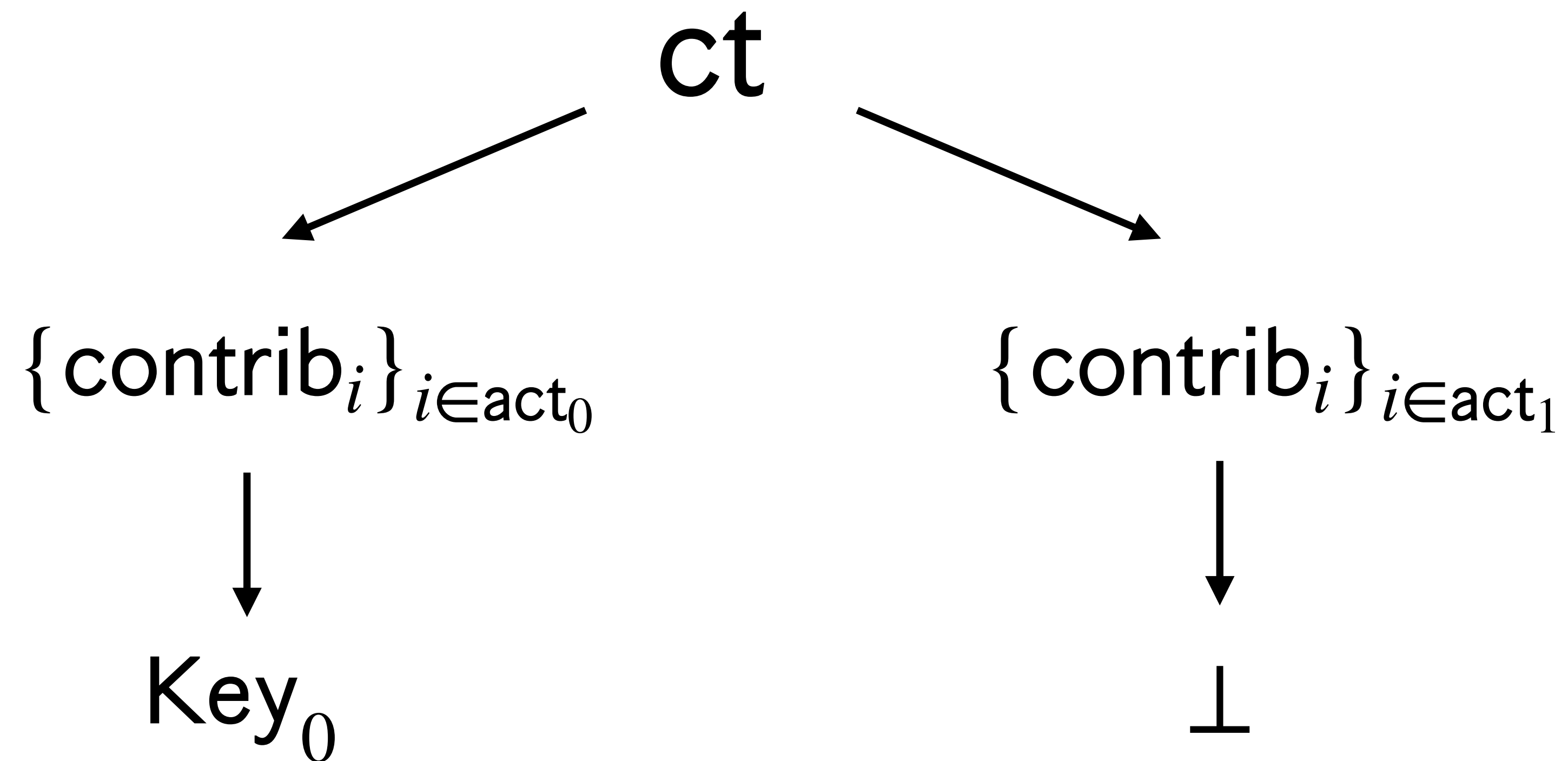
Given access to $\{dk\}_{i \in \text{corrupt}}$ and $\text{TKEM} . \text{Decaps}(\cdot)$ oracle

The Adversary returns $(ct, \{contrib_i\}_{i \in \text{act}_0}, \{contrib_i\}_{i \in \text{act}_1})$ s.t.

Decapsulation Consistency



Decapsulation Consistency



The Ciphertext: Dual Regev Encryption

$$\text{ct} = (\mathbf{u}, v), \text{ where } \mathbf{u} = \mathbf{F}_{\text{id}} \cdot s + \mathbf{e} \text{ and } v = r_{\text{id}} \cdot s + e' + \text{Encode}(\text{msg})$$

The Ciphertext

$\text{ct} = (\mathbf{u}, v)$, where $\mathbf{u} = \mathbf{F}_{\text{id}} \cdot s + \mathbf{e}$ and $v = r_{\text{id}} \cdot s + e' + \text{Encode}(\text{msg})$

$$\sum_i \text{contrib}_i = \mathbf{x} \text{ s.t. } \mathbf{x}^T \cdot \mathbf{F}_{\text{id}} = r_{\text{id}} \bmod q$$

The Ciphertext

$\text{ct} = (\mathbf{u}, v)$, where $\mathbf{u} = \mathbf{F}_{\text{id}} \cdot s + \mathbf{e}$ and $v = r_{\text{id}} \cdot s + e' + \text{Encode}(\text{msg})$

$$\sum_i \text{contrib}_i = \mathbf{x} \text{ s.t. } \mathbf{x}^T \cdot \mathbf{F}_{\text{id}} = r_{\text{id}} \bmod q$$

$$\text{msg} = \text{Round}(v - \mathbf{x}^t \cdot \mathbf{u})$$

Example Attack

$\text{ct} = (\mathbf{u}, v)$, where $\mathbf{u} = \mathbf{F}_{\text{id}} \cdot s + \mathbf{e}$ and $v = r_{\text{id}} \cdot s + e' + \text{Encode}(\text{msg})$

■ — public, ■ — sampled by the Adversary.

Example Attack

May be slightly too large.

$ct = (u, v)$, where $u = \boxed{F_{id}} \cdot \boxed{s} + \boxed{e}$ and $v = \boxed{r_{id}} \cdot \boxed{s} + \boxed{e'} + \text{Encode}(msg)$

■ — public, ■ — sampled by the Adversary.

Example Attack

May be slightly too large.

$ct = (\mathbf{u}, v)$, where $\mathbf{u} = \mathbf{F}_{id} \cdot s + \mathbf{e}$ and $v = r_{id} \cdot s + e' + \text{Encode}(msg)$

Then if $\sum_{i \in \text{act}_0} \text{contrib}_i = \mathbf{x}_0$ and $\sum_{i \in \text{act}_1} \text{contrib}_i = \mathbf{x}_1$ such that

$\mathbf{x}_0^t \cdot \mathbf{e} \ll \mathbf{x}_1^t \cdot \mathbf{e}$ we may decrypt to different messages

BCHK+ = BCHK + FO

TKEM.Keygen(1^κ)

$(\{dk_i\}_{i \in [N]}, ek) \leftarrow \text{TIBE.Setup}(1^\kappa)$
return $(\{dk_i\}_{i \in [N]}, ek)$

TKEM.Encaps(ek)

$msg \leftarrow \{0, 1\}^d$

$rand = G_{fo}(msg)$

$(sk, vk) \leftarrow \text{SIG.Keygen}(1^\kappa)$

$ct = \text{TIBE.Encrypt}(ek, id = vk, msg; rand)$

$sig \leftarrow \text{SIG.Sign}(sk, ct)$

$K = H_{fo}(msg || ct)$

return $(ct, vk, sig), K$

TKEM.Decaps($dk_i, (ct, vk, sig)$)

assert $\text{SIG.Verify}(vk, ct, sig) = 1$

$contrib_i = \text{TIBE.Extract}(dk_i, id = vk)$

return $contrib_i$

TKEM.Combine($\{contrib_i\}_{i \in \text{act}}, (ct, vk, sig)$)

assert $\text{SIG.Verify}(vk, ct, sig) = 1$

$msg = \text{TIBE.Combine}(ct, vk, \text{act}, \{contrib_i\}_i)$

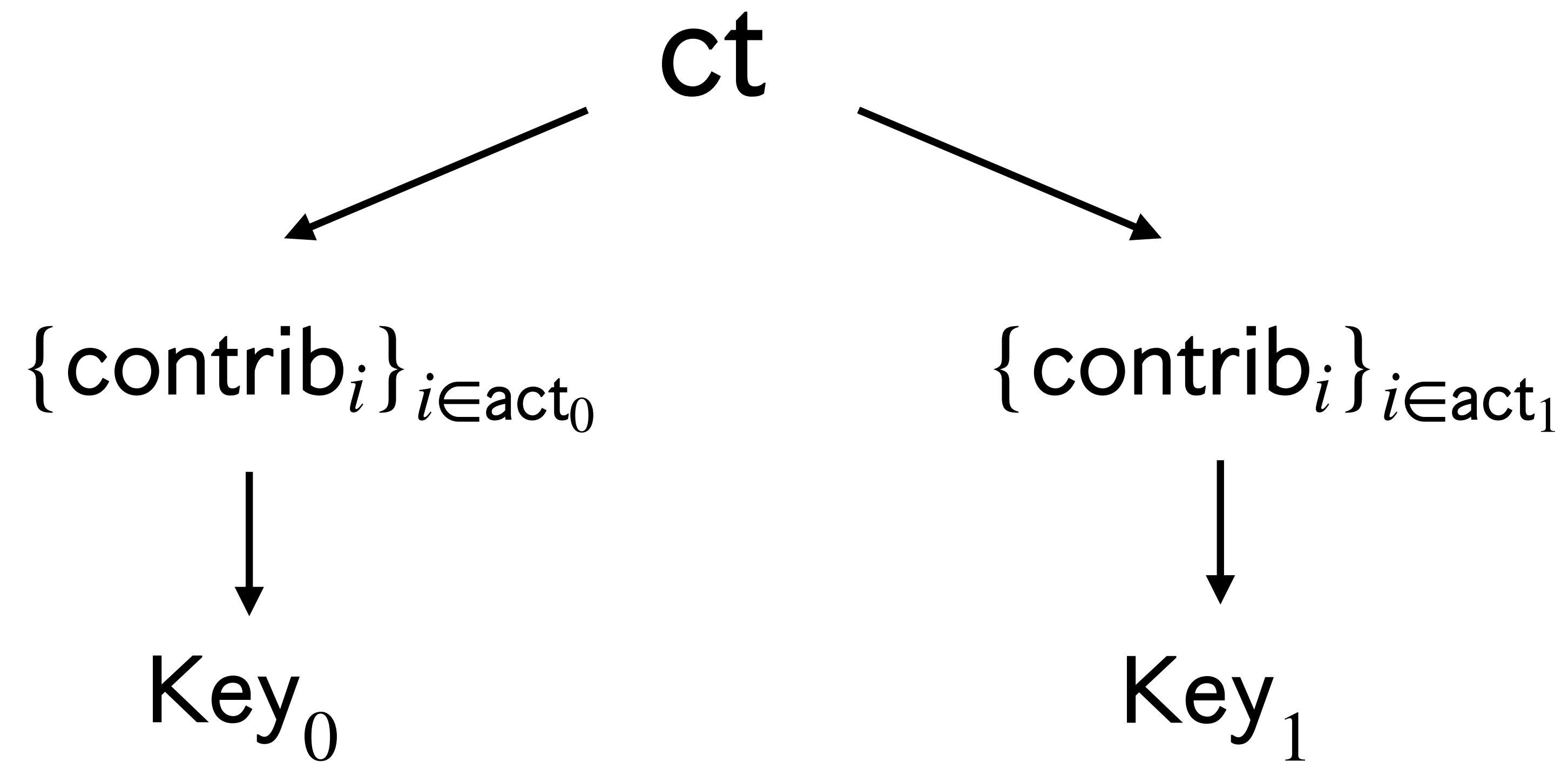
$rand = G_{fo}(msg)$

assert $ct = \text{TIBE.Encrypt}(ek, vk, msg; rand)$

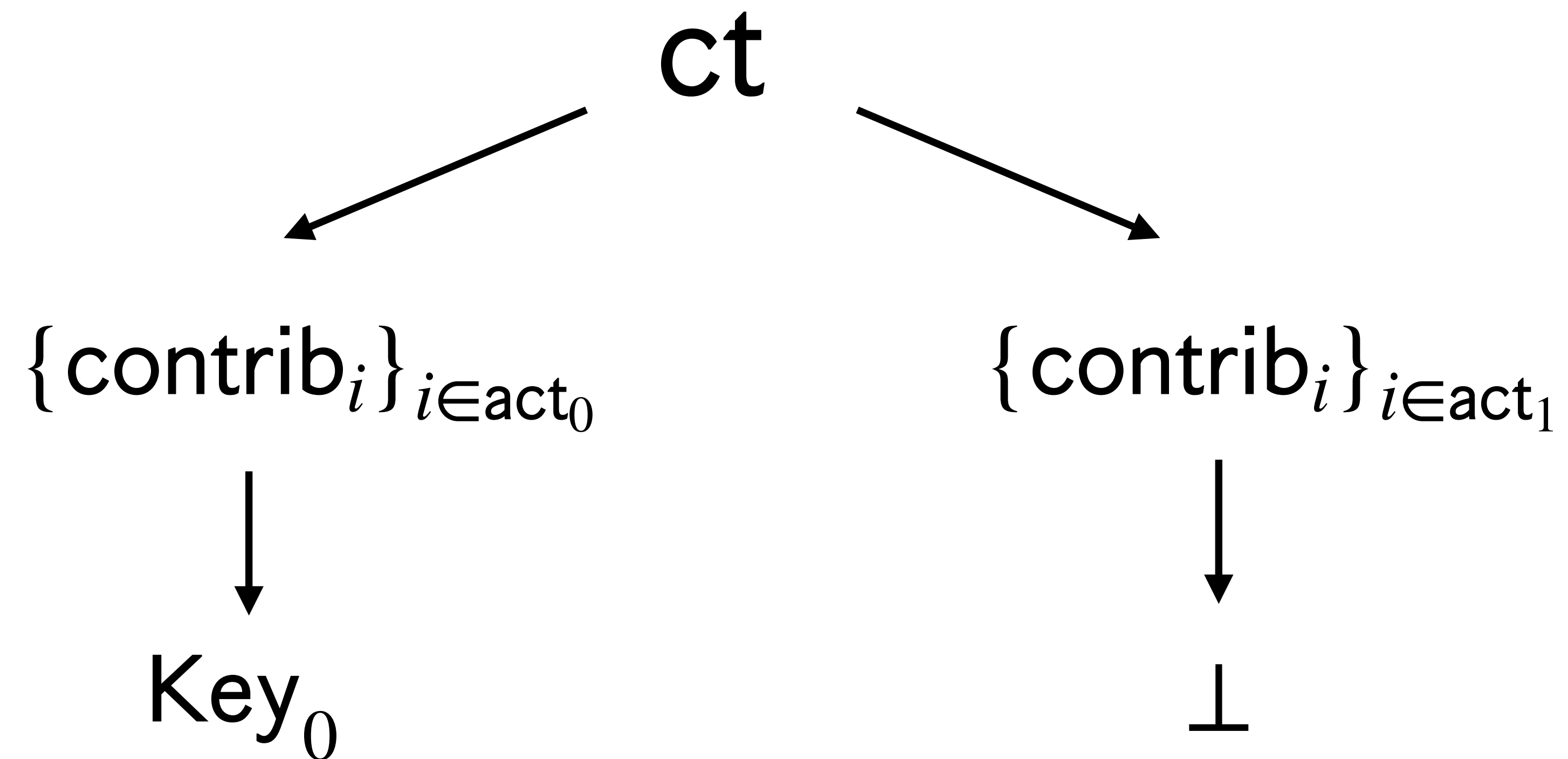
$K = H_{fo}(msg || ct)$

return K

Decapsulation Consistency



Decapsulation Consistency



Lattice-based Threshold IBE

Starting IBE

GPV IBE [GPV08]

- Dual Regev

$$\mathbf{F}_{\text{id}} = \mathbf{A}^t$$

$$r_{\text{id}} = H(\text{id})$$

- Decryption Key:

$$\text{short } \mathbf{x} \text{ s.t. } \mathbf{x}^t \cdot \mathbf{F}_{\text{id}} = r_{\text{id}} \bmod q$$

ABB IBE [ABB10]

- Dual Regev

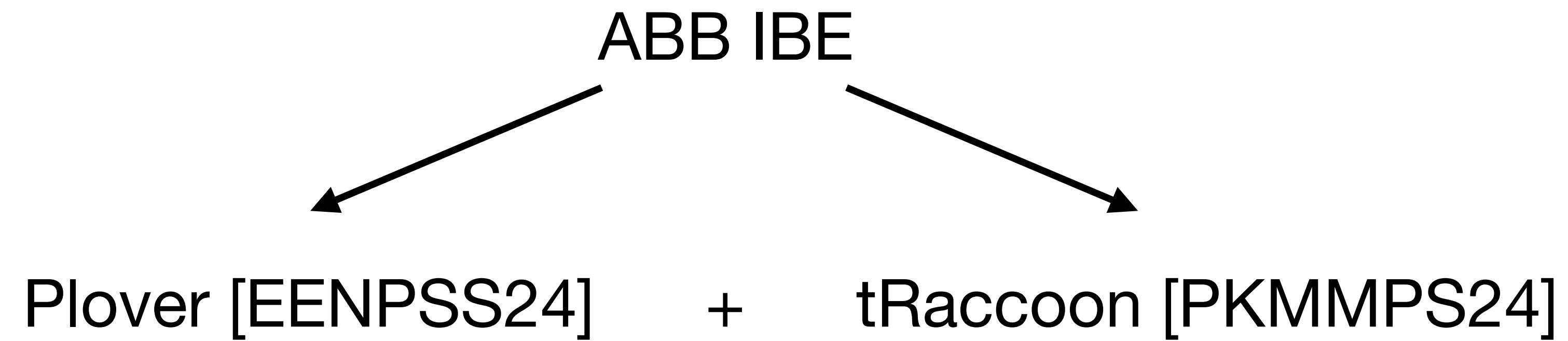
$$\mathbf{F}_{\text{id}} = [\mathbf{A}_0 \mid \mathbf{A}_1 - E(\text{id}) \cdot \mathbf{G}]^t$$

$$r_{\text{id}} = r$$

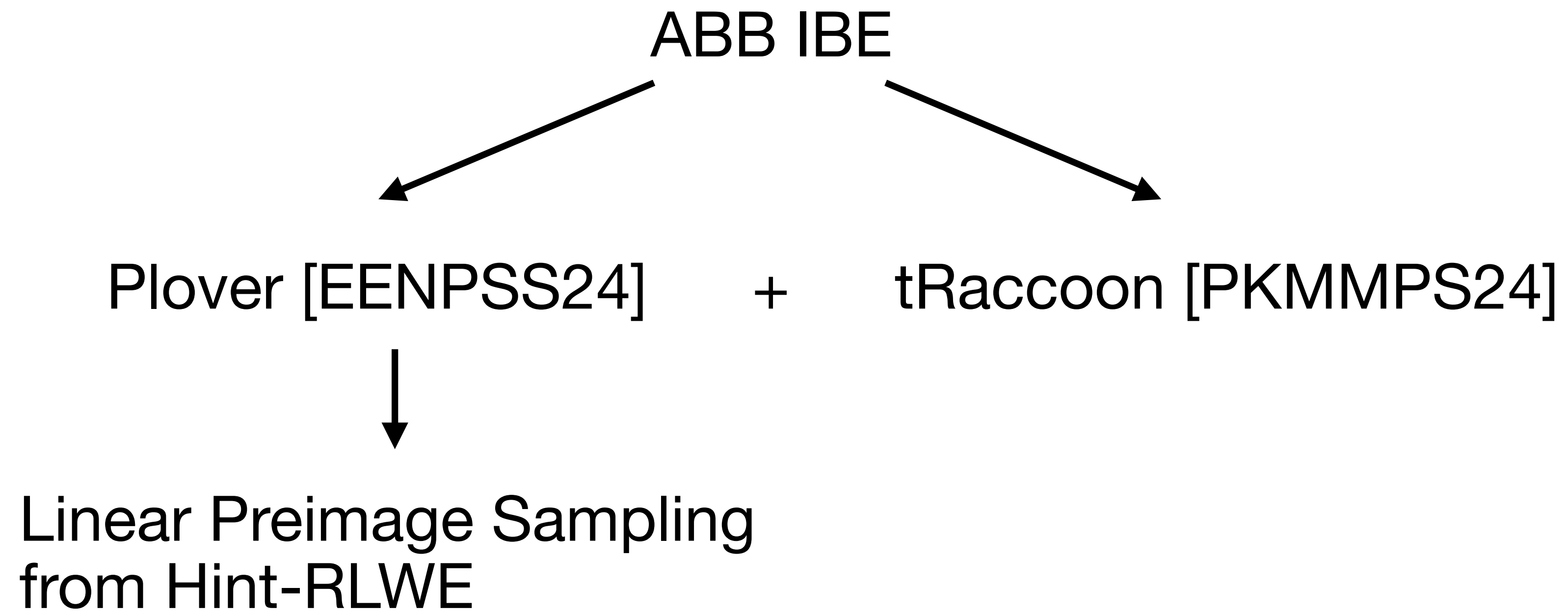
- Decryption Key:

$$\text{short } \mathbf{x} \text{ s.t. } \mathbf{x}^t \cdot \mathbf{F}_{\text{id}} = r_{\text{id}} \bmod q$$

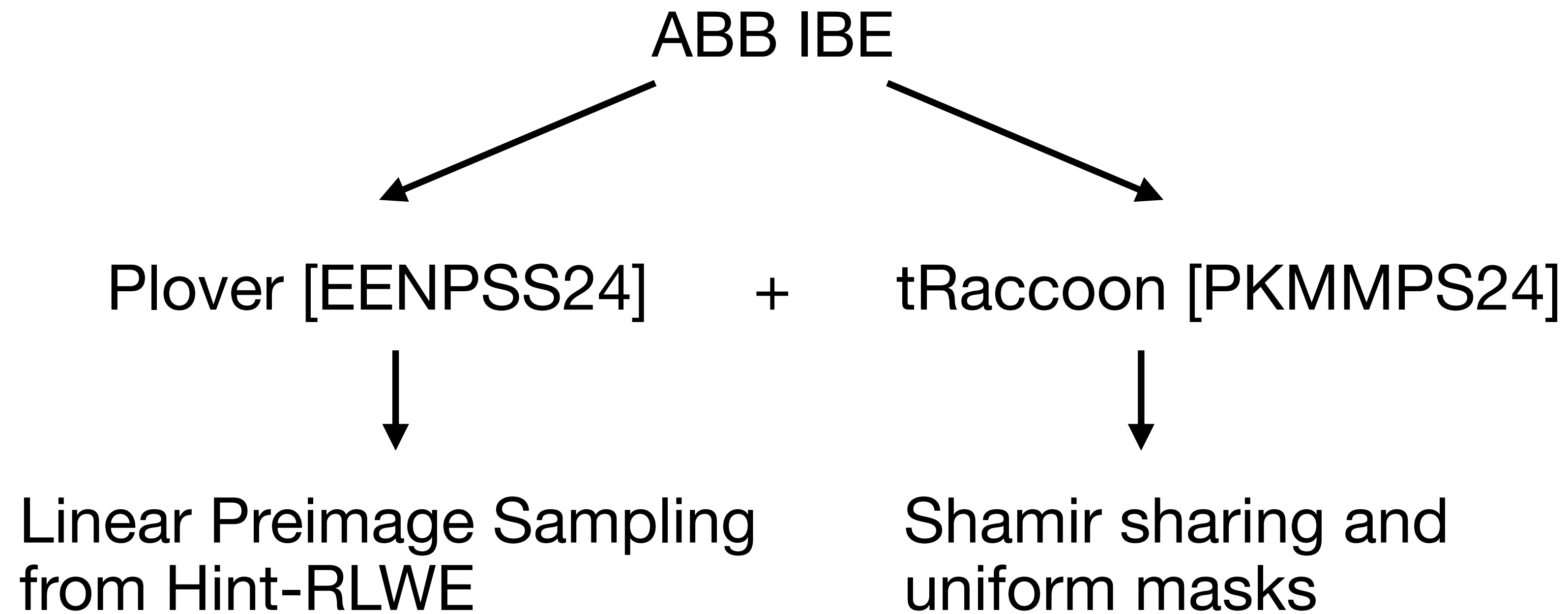
Threshold IBE



Threshold IBE



Threshold IBE



Coset-Hint-RLWE

Coset-Hint-RLWE

Let $\mathcal{R}$ be ring of integers of a cyclotomic number field, q a prime and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$

Coset-Hint-RLWE

Let $\mathcal{R}$ be ring of integers of a cyclotomic number field, q a prime and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$

Hint-RLWE Problem:

$$\{a, b = [1 \ a] \cdot \mathbf{s}, \{c_i \cdot \mathbf{s} + \mathbf{p}_i\}_{i \in Q}\} \approx \{a, b \leftarrow \mathcal{U}(\mathcal{R}_q), \{c_i \cdot \mathbf{s} + \mathbf{p}_i\}_{i \in Q}\}$$

where $a \leftarrow \mathcal{U}(\mathcal{R}_q)$, $c_i \leftarrow \chi$ and $\mathbf{s} \leftarrow \mathcal{D}_{\mathcal{R}^2, \sigma_s}$, $\mathbf{p}_i \leftarrow \mathcal{D}_{\mathcal{R}^2, \sigma_p}$

Coset-Hint-RLWE

Let $\mathcal{R}$ be ring of integers of a cyclotomic number field, q a prime and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$

Hint-RLWE Problem:

$$\{a, b = [1 \ a] \cdot \mathbf{s}, \{c_i \cdot \mathbf{s} + \mathbf{p}_i\}_{i \in Q}\} \approx \{a, b \leftarrow \mathcal{U}(\mathcal{R}_q), \{c_i \cdot \mathbf{s} + \mathbf{p}_i\}_{i \in Q}\}$$

where $a \leftarrow \mathcal{U}(\mathcal{R}_q)$, $c_i \leftarrow \chi$ and $\mathbf{s} \leftarrow \mathcal{D}_{\mathcal{R}^2, \sigma_s}$, $\mathbf{p}_i \leftarrow \mathcal{D}_{\mathcal{R}^2, \sigma_p}$

Coset-Hint-RLWE Problem:

we add $\mathbf{F} = [\mathbf{A}_0 \mid \mathbf{A}_1]$ where $(\mathbf{A}_1, \mathbf{T}_{A_1}) \leftarrow \text{TrapGen}(1^\lambda)$ and $\mathbf{p}_i \leftarrow \mathcal{D}_{\Lambda_q^{\mathbf{r}_i - f(\mathbf{s})}(\mathbf{F}), \sigma_p}$

where $f(\mathbf{s})$ ensures $\mathbf{F} \cdot \mathbf{x}_i = \mathbf{r} \bmod q$

The parameters

λ	T	Q_{Dec}	q	d	$ ek $	$ ct $
128	32	2^{40}	2^{100}	4096	50 KiB	450 KiB

λ - security parameter, T - the threshold, Q_{Dec} - number of decryption queries, q - the modulus, d - the ring degree, ek - the KEM public key, ct - the ciphertext

Open questions

- ◆ Can we improve the scheme efficiency with better desing choices?
- ◆ Is it possible to build a Threshold IBE from GPV IBE?
- ◆ Can we formalise Decryption Consistency attacks? Can they be used as a feature?

Summary

“A Lattice-Based IND-CCA Threshold KEM from the BCHK+ Transform” | ia.cr/2025/1958

- ◆ BCHK+ transform for Lattice schemes
- ◆ Lattice-based Threshold IBE
- ◆ A new assumption Coset-Hint-MLWE



References

- [12] — Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. CRYPTO 2018
- [41] — Michael Kraitsberg, Yehuda Lindell, Valery Osheter, Nigel P. Smart, and Younes Talibi Alaoui. Adding distributed decryption and key generation to a ring-LWE based CCA encryption scheme. ACISP 2019
- [24] — Julien Devevey, Benoit Libert, Khoa Nguyen, Thomas Peters, and Moti Yung. Non-interactive CCA2-secure threshold cryptosystems: Achieving adaptive security in the standard model without pairings. PKC 2021
- [20] — Kelong Cong, Daniele Cozzo, Varun Maram, and Nigel P. Smart. Gladius: LWR based efficient hybrid public key encryption with distributed decryption. ASIACRYPT 2021
- [BBH06] — Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. CT-RSA 2006
- [BBH07] — Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. SIAM J. Comput. 2007
- [CHK04] — Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. EUROCRYPT 2004
- [GPV08] — Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. STOC 2008
- [ABB10] — Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. EUROCRYPT 2010.
- [EENPSS24] — Muhammed F. Esgin, Thomas Espitau, Guilhem Niot, Thomas Prest, Amin Sakzad, and Ron Steinfeld. Plover: Masking-friendly hash-and-sign lattice signatures. EUROCRYPT 2024
- [PKMMPS24] — Rafael Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. EUROCRYPT 2024